University of Portsmouth

School of Creative Technologies

Final Year Project

How approachable is scientific and mathematical academia to game developers that have little to no formal background

by

Tayler Parsons
UP2212801

Supervisor: Jahangir Uddin

July 9, 2026

Project Type: Study

I consent to my report in this attributed format (not anonymized), subject to final approval by the Board of Examiners, being made available in the Library Dissertation Repository and/or the school digital repositories for a maximum of 10 years.

☑Yes ☐No

Modern video game development increasingly relies on complex algorithms, physics simulations, and mathematical concepts. However, it could be argued that a significant gap between contemporary research and the skills of game developers who lack a formal academic background. This report investigates the accessibility of different scientific and mathematical literature from a game development perspective. To evaluate this, this study uses different approaches such as first-hand research that was conducted through presenting developers with standard academic notation such as summation ($\sum$) and product ($\prod$) to assess their baseline comprehension. This report aims to highlight the need for more accessible academic resources to help bridge the gap between the academic and practical side of game development.

Contents

Chapter 1

Introduction

While players primarily experience gameplay and immersive worlds, these experiences are often driven - behind the scenes - by complex algorithms and mathematical concepts.

As the creative and technical vision of the project expand, the developers' reliance on advanced mathematics scales with it; whether it be to achieve more realistic physics, procedural generation, or optimize existing systems. However, when developers turn to contemporary research literature to implement or optimize these visions, they may face a critical roadblock. A significant gap exists between the formal academic notation that can be found in scientific papers, such as summation ($\sum$), or product ($\prod$) symbols, and the practical, programming-orientated skill-set of developers who may lack an existing formal background in these fields of academia. Consequently, the literature that has been created to provide a way to establish and solve problems become arguably inaccessible to a large portion of the development community.

This report aims to measure a wide-range of developers' baseline compression of different common-place concepts and notation that can be found in mathematic literature, and comparing it to the level of explanation and context that can be found in these papers.

1.1 Defining the developer

In the context of this report, a 'developer' is someone who:

1. has written code in C++ *or in a comparable game-engine language such as* C#
2. has less than **2 years** of formal further/higher-education in mathematics
3. is currently engaged in either **independent, hobbyist, or industry** game development *or currently engaged in similar fields such as software engineering or web development*

1.2 Defining the paper

In the context of this report, a 'paper' may refer to:

1. A report that is written specifically with game development in mind
2. A report that is written with physicists or mathematicians in mind
3. A report from adjacent technical fields, such as computer science, graphics programming or artificial intelligence, which uses any mathematical concepts.
4. Any peer-reviewed academic literature, journal article, or any other formal academic writing that contains contemporary physics or mathematical content.

Chapter 2

Literature Review

2.1 Introduction

Modern game engines such as Unity, Unreal, or custom engines frequently relies on advanced mathematics to achieve various tasks like linear algebra for 3D rendering. However, the academic literature that underlines these methods often presume a specific level of existing understanding in mathematics or physics. This creates a barrier-of-entry for hobbyists, indie developers, or anyone else who lack knowledge in these areas.

The purpose of this literature review is to:

(i) survey the current state of existing research of mathematical concepts often used in game development

(ii) evaluate how effectively these papers communicate these concepts to a non-specialist audience

(iii) identify any gaps that may justify the need for more practical (as in, opposed to practical)-orientated resources

2.2 Foundational Mathematics in Game Development

Table 2.1: Mathematical Concepts Used in Game Development

Concept	**Use**	**Concepts Feature In**
Linear Algebra (Vectors, Matrices, Quaternions)	Object transformations, viewport manipulation	"Quaternions, Interpolation and Animation" (Dam et al., n.d., p. 6) *3D Math Primer for Graphics and Game Development* (Dunn & Parberry, 2011)
Differential Calculus (Derivatives, Gradients)	Physics engines, procedural animation	"Real-Time Inverse Kinematics for Generating Multi-Constrained Movements of Virtual Human Characters" (Voss & Kopp, 2025, p. 3) "A Survey on SPH Methods in Computer Graphics" (Koschier et al., 2022)
Probability and Statistics	AI decision-making, procedural generation	"Learning to Generate Video Game Maps Using Markov Models" (Snodgrass & Ontañón, 2017) "Learning Probabilistic Behavior Models in Real-Time Strategy Games" (Dereszynski et al., 2011)
Vector Calculus	Particle simulations	"A Survey on SPH Methods in Computer Graphics" (Koschier et al., 2022) "Fluid Simulation with the Navier-Stokes Equations" (Wang, n.d.)

Whilst being the most popular of these concepts, Linear Algebra's most 'basic' operations such as matrix multiplications are often represented with LaTeX notation that developers must manually translate into code. Few papers, such as *Unity Artificial Intelligence Programming* (Aversa, 2022) attempt to bridge this gap by providing code examples, however the paper still assumes a level of fluency with the mathematical concepts that many indie developers may lack.

2.3 Algorithms & Accessibility

Table 2.2: Algorithms and Accessibility in Papers

Technique	**Representation**	**Alternatives**
Procedural Generation	Noise functions are expressed through the use of summation notation (e.g., $\sum P(i)$)	Unity's Perlin Noise API. Documented resources such as tutorials (e.g., "Unity3D Procedural Generated Low-Poly Terrain" (Glazycheva, 2020))
Physics Engine	Equations of motion expressed through the use of differential notation ($\partial^2 x/\partial t^2 = F/m$)	Bullet, PhysX, Jolt. Most documentation omit the fundamental mathematics. "Jolt Physics: Jolt Physics" ("Jolt Physics: Jolt Physics", n.d.)
AI Pathfinding	Expressed through Markov Chain ($\pi(\theta)$) and matrix notation	A* and Dijkstra's algorithms are very popular, and often omit the probability theory that make them operate

Whilst academic papers present the mathematics in its most unadulterated form, communities have developed standalone libraries that abstract most of the complexity. However, these libraries are rarely accompanied by the theoretical context beside it, which would help a developer understand *how* an algorithm works, which may limit their ability to expand or optimize the code.

2.4 Documentation & Communication

Table 2.3: Documentation and Communication Practices in Papers

Source	**Strengths**	**Weaknesses**
Peer-Reviewed Journals	Very Comprehensive	Very dense in notation, little to no code examples
Conferences	Primarily focuses on practical implementations, live demos	Lacks peer review, may over simplify the mathematics in sake of brevity
Industry Blogs & Tutorials	Code-focused, step-by-step explanations	Usually limited in theoretical explanation, and can be outdated
Open-Source Projects	Very accessible code, community contribution / feedback	Documentation can be inconsistent or lacking

These formats often excel at their strengths, but lack the structure to convey that which they lack. Due to this, it is quite hard to find a source that achieves to provide a structured

synthesis between the mathematical notation and code. This disconnect is what prevents the vast majority of the sources in the categories to be accessible.

2.5 How Do We Determine If a Paper is Accessible?

2.5.1 Accessibility Metric System

Definitions of Concept Approachability Dimensions

We can determine approachability for any given paper through categorising each *'dimension'* of approachability:

Table 2.4: Concept Approachability

Dimension	**Definition**	**Indicator Example**
Notation	To which degree are mathematical symbols explained or avoided	A 'notation glossary' maybe present
Clarity of Lexicon	To which degree is plain language use, and whether technical jargon is avoided	Number of technical terms in any given chapter / section of the paper
Contextualisation	To which degree do papers provide examples, diagrams, or code snippets to help provide context to the topic discussed	At least one code snippet
Pedagogy	To which degree do papers aim to teach the reader about a specific topic	

Each dimension will be scored on a 0-3 scale (0 = no accessibility, 3 = very accessible). The overall *Approachability Score* (AS) will be the mean value of the four sub-scores for that paper.

Definitions of Literature Approachability Dimensions

Table 2.5: Literature Approachability

Source	Contribution to Criteria
Unity Artificial Intelligence Programming (Aversa, 2022)	Features code comments, but expects prior knowledge of the basics of AI theory
"Quaternions, Interpolation and Animation" (Dam et al., n.d.)	Frequently uses advanced mathematical notation with very little explanatory text
"A Survey on SPH Methods in Computer Graphics" (Koschier et al., 2022)	Very dense advanced mathematical notation with few diagrams or explanatory text
"Real-Time Inverse Kinematics for Generating Multi-Constrained Movements of Virtual Human Characters" (Voss & Kopp, 2025)	Whilst still dense in mathematical notation, it provides some indicators of what some values and parts of equations mean

These studies suggest that **notation** and **contextualisation** are the most frequently neglected dimensions. However, in contrast, the *lexical clarity* tend to be adequate in industry blogs but is rarely ever present in peer-review papers.

2.5.2 Identified Gaps and What They Imply

1. **Notation Barrier:**
2. **Lack of Code Examples:**
3. **Lack of Pedagogy:**

2.6 Questionnaire

2.6.1 Designing the Questionnaire

The questionnaire will be informed by the gaps identified above. It will feature questions such as:

Table 2.6: Questionnaire Example Questions

Question	**Rationale**
How comfortable are you translating mathematical equations (e.g., $\partial^2 x/\partial t^2$) into code?	Evaluates the 'lack of code examples' gap
When reading academic papers, how often do you need to look up definitions for the symbols and concepts featured?	Evaluates the notation dimension described in table 2.4
Does the presence of pseudocode or code snippets increase your confidence in understanding a given algorithm or concept?	Validates the contextualisation dimension described in table 2.4
Do you find that visual diagrams (such as those that show coordinate spaces, or vector trajectories) are more informative than formal mathematical definitions when learning a new algorithm	Investigates the pedagogy dimension described in table 2.4
When using a physics or path-finding library, (e.g., Jolt, A*), how important is it to understand the underlying mathematical theory vs. just using the API?	Addresses the trade-offs mentioned in Section 3 regarding abstraction and theoretical contexts

Each item will use a 5-point Likert scale (1 = Not at all comfortable, 5 = Very Comfortable). The questionnaire will also collect some demographic data (such as years of coding experience, formal maths training) to determine any background variables that may influence the results.

2.6.2 Scoring System

Chapter 3

Methodology

List of Figures

List of Tables

References

Aversa, D. D. (2022, March 28). *Unity Artificial Intelligence Programming: Add powerful, believable, and fun AI entities in your game with the power of Unity*. Packt Publishing Ltd.

Dam, E. B., Koch, M., & Lillholm, M. (n.d.). Quaternions, Interpolation and Animation.

Dereszynski, E., Hostetler, J., Fern, A., Dietterich, T., Hoang, T.-T., & Udarbe, M. (2011). Learning Probabilistic Behavior Models in Real-Time Strategy Games. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, *7*(1), 20–25. https://doi.org/10.1609/aiide.v7i1.12433

Dunn, F., & Parberry, I. (2011, November 2). *3D Math Primer for Graphics and Game Development*. CRC Press.

Glazycheva, E. (2020, August 6). *Unity3D procedural generated low-poly terrain*. Medium. Retrieved July 1, 2026, from https://medium.com/@glazychevaeo/unity3d-procedural-generated-low-poly-terrain-d11914ab9f71

Jolt Physics: Jolt Physics. (n.d.). Retrieved July 1, 2026, from https://jrouwe.github.io/JoltPhysics/

Koschier, D., Bender, J., Solenthaler, B., & Teschner, M. (2022). A Survey on SPH Methods in Computer Graphics. *Computer Graphics Forum*, *41*(2), 737–760. https://doi.org/10.1111/cgf.14508

Snodgrass, S., & Ontañón, S. (2017). Learning to Generate Video Game Maps Using Markov Models. *IEEE Transactions on Computational Intelligence and AI in Games*, *9*(4), 410–422. https://doi.org/10.1109/TCIAIG.2016.2623560

Voss, H., & Kopp, S. (2025). Real-Time Inverse Kinematics for Generating Multi-Constrained Movements of Virtual Human Characters. *Proceedings of the 25th ACM International Conference on Intelligent Virtual Agents*, 1–10. https://doi.org/10.1145/3717511.3747066

Wang, A. (n.d.). Fluid Simulation with the Navier-Stokes Equations.